

DECK 00 • INTRODUCTION

Build. Ship. Verify.

A practical course in Solana development.
Six weeks. Self-paced. AI-assisted.

SUPPORTED BY
SOLANA FOUNDATION

WHAT THIS IS

A sequence of practical exercises with clear checkpoints — so you know when you're done and what to do next.

DURATION

6 weeks
self-paced

Faster if you're experienced;
slower if you're new.

EXERCISES

10 builds
+ 1 capstone

Each one deployed to devnet
and verifiable on-chain.

METHOD

AI-assisted
development

Use any tool you like. We
teach the workflow, not the
tool.

PROOF

Portfolio
on devnet

10 deployed programs and
projects you can point to.

SECTION 01

Why this course exists.

THE PROBLEM ISN'T THE RESOURCES

Most people who start learning Solana never finish. Not because the material is missing — because they don't have structure.

The problem with self-directed learning is rarely finding resources. It's knowing what order to tackle things in, what to skip, what actually matters, and how to tell whether you're making progress.

AI can answer any question you throw at it. It can't tell you which question to ask next. It can't give you a milestone and say "you should be here by now." It can't stop you from going deep on something that doesn't matter yet, while skipping something that does.



You start strong, hit something confusing in week two, and tell yourself you'll come back to it tomorrow.

Then it doesn't happen.

— THE OTHER THING THAT KILLS SELF-DIRECTED LEARNING

10

PORTFOLIO BY WEEK SIX

deployed programs & projects on devnet.

Short, completable exercises. Each one produces something concrete: a deployed program, a working script, a visible transaction on-chain. All verifiable.

DEPLOYED

DEVNET

ON-CHAIN

SECTION 02

Learning with AI.

USE AI TO MOVE FASTER

AI speeds up learning. It doesn't make it easy.

FOUR THINGS AI IS GOOD FOR

- 01 Stuck on an error.** Paste it with your code. Get a diagnosis. Skip the 45-minute Stack Overflow trawl.
- 02 A concept doesn't click.** Ask for an analogy. Ask it as if you're coming from Python. Keep questioning.
- 03 Boilerplate is boring.** Let AI generate config, scaffolding, repetitive code. Spend your time thinking.
- 04 You don't know where to start.** Describe what you want. Ask for an outline. Work the steps.



THE GOAL

The goal isn't *"get AI to write Solana code for me."* It's learning how to pick up any technical topic quickly with AI helping. You'll use that process long after this course is done.

Week 1 covers this in detail. The main idea: **use AI to move faster, but check what it gives you.**

THE FAILURE MODE

AI makes it easy to feel productive without actually learning.

Generate a program, run it, watch tests pass, understand nothing about what happened. The code works. You couldn't reproduce it.

01 DESIGN

Make decisions before you prompt.

Account model. State shape. Failure cases.

02 CHECK

Output against specific criteria.

Not just *"does it run?"*

03 VERIFY

On-chain. Not just in your terminal.

Explorer is the source of truth.

04 BUILD

Each exercise builds on the last.

You'll notice if you've missed something.

SECTION 03

How this course works.

EVERY EXERCISE FOLLOWS THE SAME STRUCTURE

Anatomy of an exercise.

- 01 Concept refresher.** A brief recap from Week 1 or earlier exercises — so you're not flipping between documents.
- 02 What you're building.** A clear description of the end product. What the program does. What the script outputs. What the UI shows.
- 03 Design considerations.** Questions to think through before you start coding. Decisions AI can't make for you.
- 04 Building it.** Step-by-step guidance — without giving you the code. You and your AI tools write it.
- 05 You know it worked when...** A self-verification checklist. Observable, concrete outcomes. No ambiguity.
- 06 Stretch goal.** Optional. Harder. Less guided. Often previews concepts from later weeks.

YOU'LL SEE COLOURED BOXES THROUGHOUT

Three colors. Three jobs.



PURPLE • TASKS & INSTRUCTIONS

Step-by-step actions to complete, or a list of things to include in your build. *Do these.*



BLUE • GUIDANCE & LEARNING PROMPTS

Suggestions for how to approach a concept, what to ask AI about, or how to think through a design decision.



AMBER • WARNINGS & PITFALLS

Things likely to go wrong: common AI mistakes, version issues, debugging tips, patterns that trip people up.

AI SUPPORT TAPERS WEEK BY WEEK

More scaffolding early. Less by the capstone.

WEEK	AI SUPPORT LEVEL	WHAT THAT MEANS
01	GUIDED EXPLORATION	Learn how to use AI for technical learning. Practice the verify loop. <i>No coding.</i>
02	HEAVY SCAFFOLDING	Exercises include prompting guidance, output-checking walkthroughs, and common-error tables.
03	LIGHTER SCAFFOLDING	Exercises flag what to watch for, but give you more room to figure things out.
04	MINIMAL	Pitfall warnings only. No prompting guidance. You manage your own workflow.
05–06	NONE	You decide when and how to use AI. The skill is yours.

SIX WEEKS, IN ORDER

Course outline.

WEEK	TOPIC	WHAT YOU’LL BUILD / LEARN
01	Foundations	Solana architecture, accounts model, PDAs, tokens, dev environment overview. How to learn with AI.
02	First deployments	Ex 1: Set up environment, deploy your first Anchor program to devnet. Ex 2: Counter program with PDAs + TypeScript client.
03	Tokens	Ex 3: SPL token - mint, transfer, balance script. Ex 4: Token-2022 mint with extensions, decode extension data.
04	Programs & frontend	Ex 5: Voting program (state machine). Ex 6: Tip jar with CPI. Ex 7: React frontend + wallet-adapter.
05	Escrow, DeFi & NFTs	Ex 8: Token escrow (make/take/cancel). Ex 9: Pyth prices + Jupiter quotes. Ex 10: NFT collection with Metaplex Umi.
06	Capstone	Combine 3+ skills from previous exercises. Deployed program, working frontend or CLI, documented README.

FROM PEOPLE WHO'VE DONE THIS BEFORE

Five rules to keep you moving.

01

Do the exercises in order.

They build on each other. Ex 7 uses the program from Ex 5. Ex 8 uses tokens from Ex 3.

02

Don't skip the failure tests.

Wrong authority. Invalid state. Double-spend. That's where you learn what your program enforces.

03

Read AI output before you run it.

If you can't explain what a line does, ask AI to explain it before you move on.

04

Verify on-chain.

Look up every artefact on Explorer. You'll catch things in Explorer you'd miss in your terminal.

05

Keep going when you get stuck.

Stuck for 30 minutes? Take a break, come back. Fresh eyes help. You will get stuck. That's normal.

11 ARTEFACTS, ALL ON DEVNET

Everything you build is verifiable on-chain.

01	Working dev environment configured for devnet	SETUP	07	A React frontend connected to your on-chain programs	EX 07
02	An Anchor program that reads & writes PDA state	EX 02	08	A fully functional escrow for trustless token exchange	EX 08
03	An SPL token you created, minted, and transferred	EX 03	09	A DeFi tool reading Pyth prices and Jupiter quotes	EX 09
04	A Token-2022 mint with custom extensions	EX 04	10	An NFT collection with on-chain metadata	EX 10
05	A state-machine program with guarded transitions	EX 05	11	A capstone project combining multiple skills	CAPSTONE
06	A tip jar demonstrating CPI & PDA signing	EX 06			